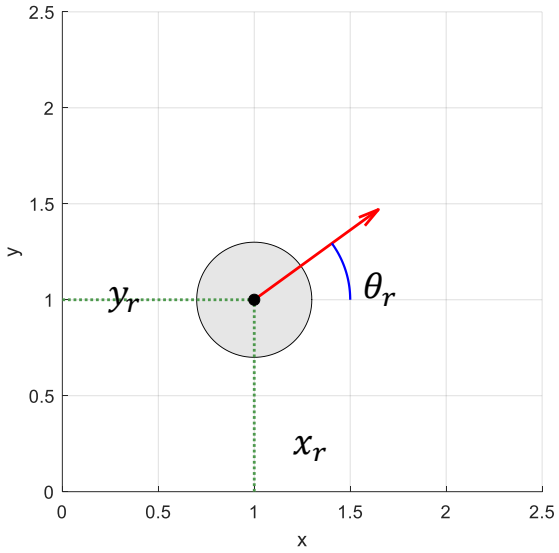
Zadanie 3 . 2D Lokalizácia mobilného robota s využitím LIDARU – algoritmus ICP

Optimalizácia a optimálne riadenie

- Robot s diferenciálnym podvozkom sa pohybuje v 2D priestore



- Budeme uvažovať nelineárny model systému v tvare

$$\begin{pmatrix} x_r(k+1) \\ y_r(k+1) \\ \theta_r(k+1) \end{pmatrix} = \begin{pmatrix} x_r(k) + v_r(k)\Delta t \cos(\theta_r(k)) \\ y_r(k) + v_r(k)\Delta t \sin(\theta_r(k)) \\ \theta_r(k) + \omega(k)\Delta t \end{pmatrix} + \begin{pmatrix} w_1(k) \\ w_2(k) \\ w_3(k) \end{pmatrix}$$

  - x_r, y_r sú súradnice robota, θ_r je jeho orientácia (natočenie)
  - v_r je dopredná rýchlosť pohybu, ω je uhlová rýchlosť otáčania (vstupy)
  - Δt je diskrétny časový krok

Cieľom je odhadnúť neznámu pozíciu $x_r(k)$, $y_r(k)$ a natočenie robota $\theta_r(k)$

- Uvažujme dva súradnicové systémy podliehajúce transformácii (rotácia + translácia)

$$p_B(\theta, t) = R(\theta)p_A + t$$

kde $R(\theta) = R_z(\theta) \in \mathbb{R}^{2\times 2}$ je rotačná matica s natočením $\theta = \theta_r$ v osi Z a $t \in \mathbb{R}^2$ je vektor translácie.

Medzi stavovým vektorom a transformáciou súradnicových systémov platí $\theta = \theta_r$ a $t = \begin{pmatrix} x_r \\ y_r \end{pmatrix}$

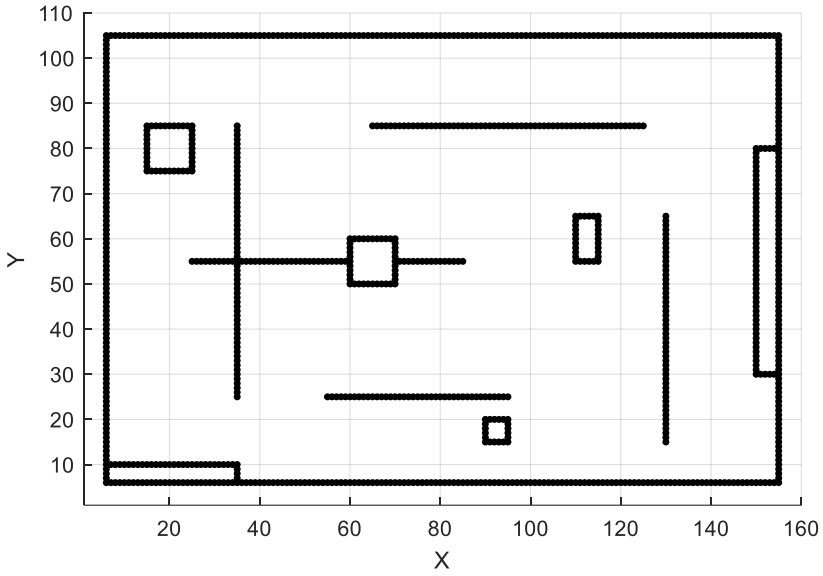
  - p_B – globálny súradnicový systém
  - p_A -súradnicový systém robota

- Neznáme parametre transformácie sú orientácia $\theta \in \mathbb{R}$ a translačný vektor $t \in \mathbb{R}^2$
- Uvažujme N bodov $p_{A_1}, p_{A_2}, p_{A_3} \dots p_{A_N}$ a $p_{B_1}, p_{B_2}, p_{B_3} \dots p_{B_N}$

$$P_A = \begin{pmatrix} p_{A_1} \\ p_{A_2} \\ \vdots \\ p_{A_N} \end{pmatrix}, P_B = \begin{pmatrix} p_{B_1} \\ p_{B_2} \\ \vdots \\ p_{B_N} \end{pmatrix}$$

Robot má 2D LIDAR ktorý v každej perióde vzorkovania poskytuje mračno bodov P_A v súradnicovom systéme robota.

Množinu všetkých existujúcich referenčných bodov P_B budú tvoriť prvky známej okupačnej mapy v ktorej sa robot pohybuje



- Definujeme účelovú funkciu ako súčet veľkostí rozdielov vektorov

$$J(\theta, t) = \frac{1}{2}\sum_{i=1}^{N}\left(R(\theta)p_{A_i} + t - p_{B_i}\right)^T\left(R(\theta)p_{A_i} + t - p_{B_i}\right)$$

- Tento optimalizačný problém riešime ako nelineárnu metódu najmenších štvorcov

$$J(x) = \frac{1}{2}(f(x) - y)^T(f(x) - y)$$

kde vektor optimalizovaných premenných $x \in \mathbb{R}^3$ bude

$$x = \begin{pmatrix} \theta \\ t \end{pmatrix}$$

$$J(\theta, t) = \frac{1}{2}\left(\begin{pmatrix} R(\theta) & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & R(\theta) & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & R(\theta) \end{pmatrix} P_A + \begin{pmatrix} I \\ I \\ \vdots \\ I \end{pmatrix} t - P_B\right)^T\left(\begin{pmatrix} R(\theta) & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & R(\theta) & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & R(\theta) \end{pmatrix} P_A + \begin{pmatrix} I \\ I \\ \vdots \\ I \end{pmatrix} t - P_B\right)$$

- Na minimalizáciu účelovej funkcie (odhad transformácie) použijeme Gauss-Newtonov algoritmus

$$x(k+1) = x(k) - \left[\left(\nabla f(x)_{x=x(k)}\right)^T\left(\nabla f(x)_{x=x(k)}\right) + \Lambda\right]^{-1}\left(\nabla f(x)_{x=x(k)}\right)^T\left(f\left(x(k)\right) - y\right)$$

- Pričom teda gradient a Hessián sú:

$$\nabla J(x) = \left(\nabla f(x)\right)^T(f(x) - y) = \left(\nabla f(x)\right)^T r(x)$$

$$\nabla^2 J(x) = \left(\nabla f(x)\right)^T \nabla f(x)$$

- Algoritmus môžeme ukončiť ak $\nabla J(x)^T \nabla J(x) < \epsilon$ teda ak

$$\left(f(x(k)) - y\right)^T \left(\nabla f(x)_{x=x(k)}\right)\left(\nabla f(x)_{x=x(k)}\right)^T \left(f(x(k)) - y\right) < \epsilon$$

- Jakobián funkcie $f(x)$ je daný ako

$$\nabla f(\theta, t) = \begin{pmatrix} \frac{\partial f(\theta,t)}{\partial \theta} & \frac{\partial f(\theta,t)}{\partial t} \end{pmatrix}$$

$$\frac{\partial f(\theta,t)}{\partial \theta} = \begin{pmatrix} \frac{\partial R(\theta)}{\partial \theta} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \frac{\partial R(\theta)}{\partial \theta} & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \frac{\partial R(\theta)}{\partial \theta} \end{pmatrix} P_A$$

$$\frac{\partial f(\theta,t)}{\partial t} = \begin{pmatrix} I \\ I \\ \vdots \\ I \end{pmatrix}$$

Rotačná matica a jej derivácia sú

$$R_z(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \frac{\partial R_z(\theta)}{\partial \theta} = \begin{pmatrix} -\sin(\theta) & -\cos(\theta) & 0 \\ \cos(\theta) & -\sin(\theta) & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Algoritmus ICP hľadá korešpondencie (páry bodov) medzi LIDAR skenom (mračnom bodov) a bodmi známej mapy prostredia.

Kroky algoritmu ICP

1. Inicializácia: Začni s počiatočným odhadom θ a t

2. Priradenie: Pre každý bod p_{A_i}, nájdi najbližší bod p_{B_i} podľa euklidovskej vzdialenosti a preusporiadaj body p_{A_i}

3. Odhadni transformáciu: Nájdi optimálnu rotáciu a transláciu $p_B(\theta, t) = R(\theta)p_A + t$, ktorá najlepšie zarovná páry p_{A_i}, p_{B_i} pomocou nelineárnej metódy najmenších štvorcov.

4. Aplikuj transformáciu: Posuň zdrojové body p_{A_i} pomocou vypočítaného $R(\theta)$ a t.

Opakuj: Kroky 2–4 opakuj, až kým dosiahneš požadovanú toleranciu.

EKF pre spresnenie odhadu stavu

- Stavový vektor je potrebné odhadnúť (zrekonštruovať) na základe:
    - Modelu systému
    - Meraní výstupov
    - Informácie o vstupoch
- Prechodová funkcia $f\left(x(k), u(k)\right)$ bude

$$f\big(x(k),u(k)\big)=\begin{pmatrix}x_r(k)+v_r(k)\Delta t\cos(\theta_r(k))\\ y_r(k)+v_r(k)\Delta t\sin(\theta_r(k))\\ \theta_r(k)+\omega(k)\Delta t\end{pmatrix}$$

Funkcia merania bude (algoritmus ICP poskytuje zašumený odhad $t=\begin{pmatrix}x_r\\ y_r\end{pmatrix}, \theta=\theta_r$)

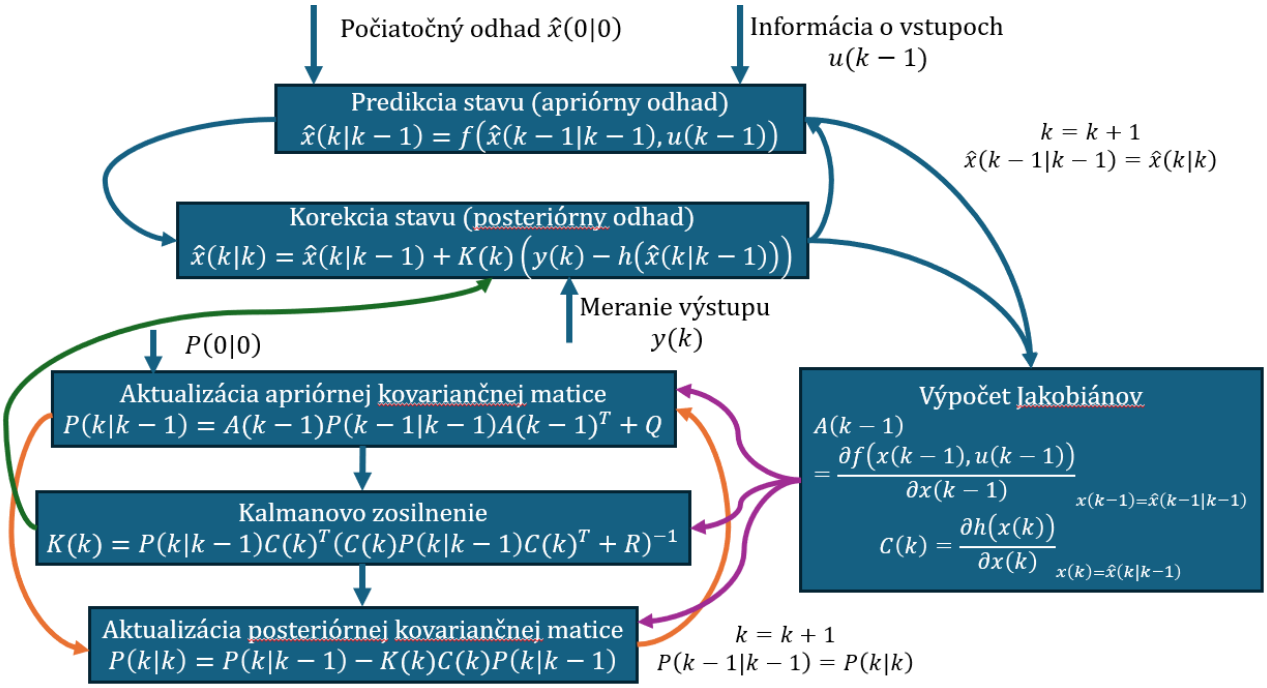
$$h\big(x(k)\big)=\begin{pmatrix}x_r\\ y_r\\ \theta_r\end{pmatrix}$$

- Potrebujeme určiť Jakobiány prechodovej funkcie $\frac{\partial f(x(k),u(k))}{\partial x(k)}$ a funkcii merania $\frac{\partial h(x(k))}{\partial x(k)}$

$$A(k)=\frac{\partial f\big(x(k),u(k)\big)}{\partial x(k)}=\begin{pmatrix}1 & 0 & -v_r(k)\Delta t\sin(\theta_r(k))\\ 0 & 1 & +v_r(k)\Delta t\cos(\theta_r(k))\\ 0 & 0 & 1\end{pmatrix}$$

$$C(k)=\frac{\partial h\big(x(k)\big)}{\partial x(k)}=\begin{pmatrix}1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & 1\end{pmatrix}$$

Algoritmus rozšíreného Kalmanovho filtra:



Použite skript *lidar_scr*

Implementujte funkciu pre výpočet rotačnej matice v osi Z (priečinok geometry)

```
function [Rz] = rotZ(theta)
%TODO
Rz=[];
```

```
end
```

Implementujte funkciu pre výpočet derivácie rotačnej matice v osi Z (priečinok geometry)

```
function [dRz] = drotZ(theta)
%TODO
dRz = [];
end
```

Implementujte Gauss-Newtonov algoritmus (priečinok optim_alg)

```
function [x_hat, g, H] = optim_GaussNewton(f, x_hat, tol,Lambda)

while 1
    [r, J] = f(x_hat);
    H = %TODO
    g = %TODO
    if g' * g < tol
        break;
    end
    delta = %TODO
    x_hat = %TODO
end

end
```

Implementujte funkciu pre odhad transformácie (priečinok ICP) a pomocnú funkciu pre výpočet reziduí a ich Jakobiánu

```
function [theta_hat,t_hat,H] =
transform_estimate2D(pA,pB,theta_hat,t_hat,tol,Lambda)

[x_hat,~,H] = optim_GaussNewton(@(x_hat)get_rJ(x_hat,pA,pB),[theta_hat;t_hat],tol,
Lambda);

theta_hat=x_hat(1);
t_hat=x_hat(2:3);
end

function [r,J] = get_rJ(x_hat,pA,pB)

theta_hat=x_hat(1);
t_hat=x_hat(2:3);
N=size(pA,1);
R_hat=%TODO
d_R_hat=%TODO
r = %TODO
df_d_theta=%TODO
J=%TODO
end
```

Implementujte funkciu pre algoritmus ICP

Pozn. volanie `[idx, d] = knnsearch(KDtree, pA_transformed,'k',1)` realizuje efektívne hľadanie najbližších susedov medzi transformovanými bodmi mapy $R(\theta)p_{A_i} + t$ a dátami z Lidaru p_{B_i} s využitím takzvaného KD stromu.

KD strom sa vytvára so známej mapy volaním

```
KDtree = KDTreeSearcher(pB);


function [theta_hat, t_hat] = ICP_estimate_2D(pA, pB, theta_hat, t_hat, tol,
tol_d, KDtree, Lambda)
i=0;
    while true
      R_hat = %TODO
      pA_transformed = %TODO
      [idx, d] = knnsearch(KDtree, pA_transformed,'k',1);
      [theta_hat, t_hat, ~] = transform_estimate2D(%TODO, theta_hat, t_hat, tol,
Lambda);

        i=i+1;
        if mean(d) < tol_d || i>200
            break;
        end
    end

end
```

Použitie EKF

Skript *lidar.scr* využíva rozšírený Kalmanov filter (EKF) na odhad stavu robota na základe výstupu ICP algoritmu (merania)

Implementujte predikciu stavu, kovariančnej matice a výpočet stavového Jakobiánu

```
A=%TODO
P=%TODO
x_hat=%TODO
```

Implementujte výstupný Jakobián, výpočet Kalmanovho zosilnenia, korekciu stavu meraním a korekciu kovariančnej matice

```
y=%TODO
C=%TODO
y_hat=%TODO
K=%TODO
x_hat=%TODO
P=%TODO
```

Riešenie

